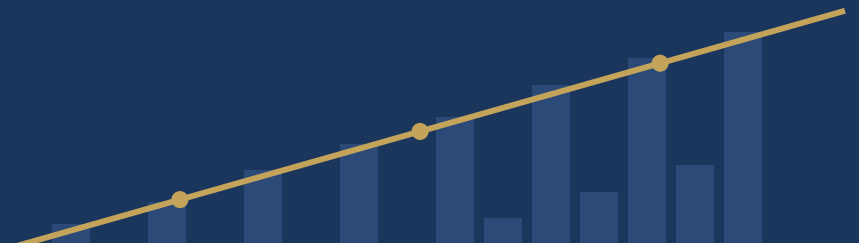


Predictive Insight Engine

Theory Concept Notes

Real Estate House Price Prediction

Simple · Multiple · Polynomial Regression + Gradient Descent



WHAT THESE NOTES COVER

Parts A-I of the project: conceptual foundations, model mathematics, evaluation metrics, bias-variance diagnostics, and the three flavours of gradient descent — written as study notes, not as code. Use them to write theory answers, viva explanations, and the final report.

00

How to use these notes

A map from assignment parts to theory chapters

This booklet is the **theory companion** to the Predictive Insight Engine project. The notebook implements models in Python; these notes explain what those models are, why they work, which assumptions they rest on, and how to talk about results in a viva, report, or exam. Every chapter is tied back to the house-price task so the maths never floats free of the business problem.

Read **Chapters 1-4** for Part A (conceptual understanding). Use **Chapters 5-8** while writing Parts B-E. Use **Chapters 9-11** for polynomial models, bias-variance, and gradient descent. Chapter 12 is a one-page formula sheet; Chapter 13 is a viva crib.

Contents

Ch.	Topic	Assignment map
01	Supervised Learning Algorithms	Part A · Q1
02	Regression vs Classification	Part A · Q2
03	Simple Linear Regression	Parts A, C · Q3, Q10-12
04	Assumptions of Linear Regression	Part A · Q4; Part C · Q12
05	Data, Features & Train-Test Split	Part B · Q7-9
06	Multiple Linear Regression	Part E · Q15-17
07	Polynomial Regression	Part F · Q18-20
08	Model Evaluation Metrics	Part D · Q13-14
09	Bias-Variance, Overfitting & Underfitting	Parts A, H · Q5-6, Q26-28
10	Gradient Descent Optimisation	Part G · Q21-25
11	Diagnostics, Comparison & Business Meaning	Part I · Q29
12	Formula Sheet & Glossary	Revision
13	Viva / Exam Quick Answers	Parts A-I

In this project — the one-sentence brief

You are a junior data scientist at a real-estate analytics firm. Given ~4,200 labelled houses, you must predict **house_price_inr** from structural and location features, compare Simple, Multiple and Polynomial Regression, implement Gradient Descent from scratch, and explain **why** one model generalises better than the others.

01

Supervised Learning Algorithms

Learning a mapping from labelled examples

1.1 What “supervised” actually means

A **supervised learning** algorithm is given a dataset of input-output pairs $(\mathbf{x}_i, \mathbf{y}_i)$. The output $\mathbf{y}_i$ is called the **label** or **target**. The algorithm’s job is to learn a function $\mathbf{f} : \mathbf{X} \rightarrow \mathbf{Y}$ such that $\mathbf{f}(\mathbf{x})$ is close to the true $\mathbf{y}$, not only on the training houses but on houses the model has never seen.

The word **supervised** is used because a “teacher” (the historical data) already knows the correct answer during training. Contrast this with **unsupervised learning** (clustering, PCA — no target column) and **reinforcement learning** (an agent learns from rewards, not labelled rows).

Supervised Learning Pipeline



Supervision = the target $\mathbf{y}$ is known during training. Goal: generalise to new $\mathbf{X}$.

Figure 1. Supervised pipeline: labelled data → training algorithm → frozen model → prediction on new $\mathbf{X}$.

1.2 The three ingredients

Ingredient	Symbol	In the house-price project
Features / inputs	$\mathbf{X}$	area_sqft, bedrooms, bathrooms, location_score, age_years, ...
Target / label	$\mathbf{y}$	house_price_inr (continuous rupees)
Hypothesis / model	$\mathbf{f}$ or $\mathbf{h}_\theta$	a linear, polynomial, or GD-fitted function
Loss / cost	$J(\theta)$	usually Mean Squared Error of predicted vs actual price
Parameters	θ or β	intercept and feature weights the algorithm estimates

1.3 Training versus inference

Training (also called fitting) uses the training set to choose parameters that minimise the cost. **Inference** (prediction) applies the frozen parameters to a new feature vector. A model that merely memorises training prices is useless to a realtor — the whole point is **generalisation** to unseen listings.

Typical supervised algorithms: Linear / Polynomial / Ridge / Lasso Regression, Logistic Regression, Decision Trees, Random Forests, Support Vector Machines, k-Nearest Neighbours, Gradient Boosting, Neural Networks. This project focuses on the linear family because it is interpretable — a

firm can tell a client which features drive price and by how much.

Exam-ready definition

Supervised learning algorithms learn a mapping from labelled input-output pairs ($X \rightarrow y$). The model is trained where the correct target is known, then used to predict that target on unseen data. House-price prediction is supervised because every training house already has a known sale/list price.

02

Regression vs Classification

The two supervised families

Once a problem is supervised, the next fork is the **type of the target y** . This single fact decides the model family, the loss, and the metrics.

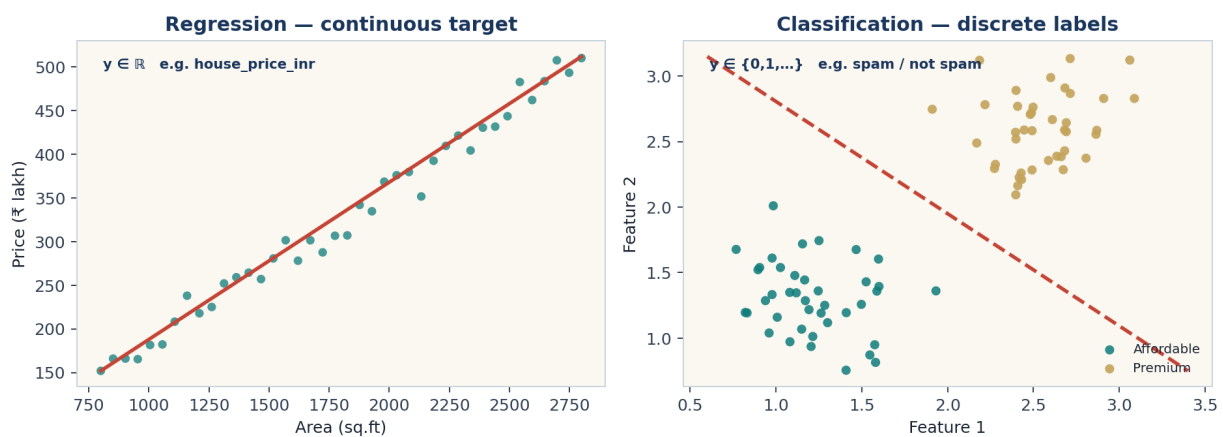


Figure 2. Regression predicts a number on a continuous scale; classification predicts a class label.

	Regression	Classification
Target y	Continuous number (real-valued)	Discrete class / category
Example	House price in ₹	Spam vs not-spam; premium vs affordable
Output	$\hat{y} = 42,50,000$	$\hat{y} = \text{"premium"} \text{ or class probability}$
Typical loss	MSE, MAE, Huber	Log-loss, Gini, hinge, cross-entropy
Typical metrics	RMSE, MAE, R^2 , Adj. R^2	Accuracy, Precision, Recall, F1, AUC
Decision surface	A fitted curve / hyperplane through numbers	Boundaries that separate classes
This project?	Yes — house_price_inr	No (unless we binned prices)

2.1 Why this project is regression — and why that matters

A house price of ₹41,20,000 is meaningfully different from ₹41,80,000. The target lives on a continuous rupee axis, so we need models that output a real number and metrics that measure rupee error (MAE, RMSE), not “percentage of houses labelled correctly”.

If the firm instead asked “will this listing sell above ₹50 lakh — yes or no?”, that would be **binary classification**. Same features, different y , different algorithm. Never mix the two families in an

answer: Linear Regression is not a classifier; Logistic Regression is not a price predictor (it predicts a probability).

Memory hook

Regression = how much? (a number) | **Classification = which group?** (a label). If you can take a meaningful average of y , it is regression. You can average prices; you cannot average “spam” and “not spam”.

03

Simple Linear Regression

One feature, one straight line, ordinary least squares

3.1 The model

Simple Linear Regression (SLR) models the relationship between **one** independent variable x and a dependent variable y as a straight line. In this project x is typically **area_sqft** — the feature with the strongest linear association with price.

$$y = \beta_0 + \beta_1 x + \varepsilon$$

β_0 is the **intercept** (predicted price when area = 0 — a theoretical baseline, not a real house). β_1 is the **slope** (rupees of price change for a 1 sq.ft increase in area, holding nothing else because nothing else is in the model). ε is the **irreducible error** — everything the line cannot explain (negotiation, interior quality, luck).

$$\hat{y} = \beta_0 + \beta_1 x$$

The hat on $\hat{y}$ means “estimated / predicted”. We never observe the true β ; we estimate it from data.

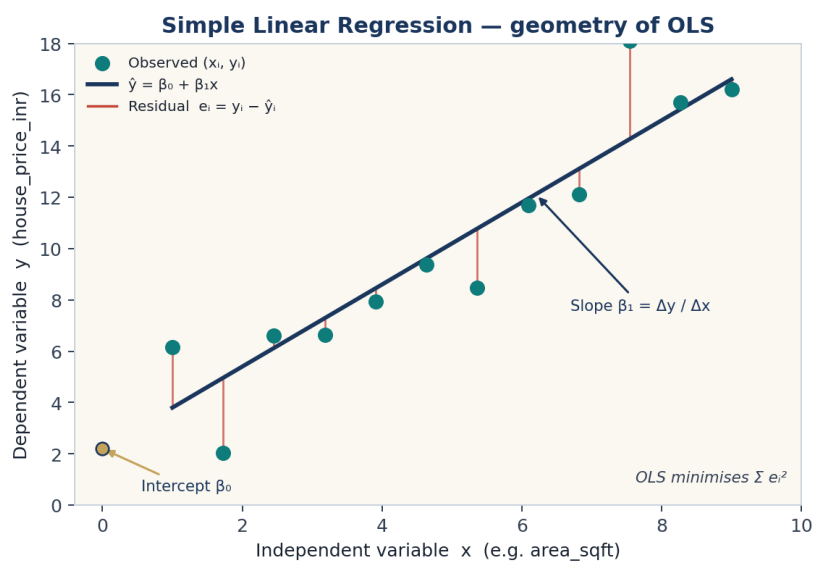


Figure 3. Geometry of OLS: the line is placed so the sum of squared vertical residuals is smallest.

3.2 Ordinary Least Squares (OLS)

A residual is $e_i = y_i - \hat{y}_i$. Squaring it punishes large misses more than small ones and makes the algebra differentiable. OLS chooses β_0, β_1 to minimise the residual sum of squares:

$$J(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

The closed-form solution (no iteration needed) is:

$$\beta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

Intuition: β_1 is covariance(x, y) / variance(x). If area and price move together strongly, the slope is steep. The intercept then shifts the line so that it passes through the point of means $(\bar{x}, \bar{y})$.

3.3 Interpreting slope and intercept in rupees

Quantity	Meaning in this project
Slope β_1	Holding the model as-is, each extra square foot of area changes predicted price by β_1 rupees.
Intercept β_0	Predicted price of a house with area = 0. Rarely meaningful on its own; it is a mathematical baseline.
Sign of β_1	Positive: larger homes cost more (expected). Negative would be a data or specification problem.
Units	β_1 is ₹ per sq.ft; β_0 is ₹.

Limitation of SLR

A house is not only its area. Two 1,200 sq.ft flats — one in a high location_score neighbourhood with a pool, one old and far from the city — should not share the same predicted price. SLR is a teaching baseline. It usually **underfits** (high bias) on this dataset.

04

Assumptions of Linear Regression

When the straight-line story is statistically valid

OLS always produces a line. The line is a **Best Linear Unbiased Estimator (BLUE)** — and the usual p-values / confidence intervals are trustworthy — only if the classical assumptions hold. Remember them as **LINE + M**.

#	Assumption	Plain English	If violated
L	Linearity	$E[y X]$ is a linear function of the features. Scatter of y vs x (or residuals vs $\hat{y}$) shows no curve.	Fit is systematically wrong; try polynomials or transforms.
I	Independence	One house's error does not inform another's (no clustering, no time autocorrelation).	Standard errors are wrong; inference is over-confident.

#	Assumption	Plain English	If violated
N	Normality of residuals	ε is roughly Gaussian. Needed for small-sample inference, not for unbiasedness of β .	Predictions can still be fine; tests/CIs less reliable.
E	Equal variance (homoscedasticity)	Spread of residuals is constant across $\hat{y}$. No funnel.	OLS is inefficient; RMSE misleads for expensive houses.
M	No (perfect) multicollinearity	Features are not linear combinations of each other. Only relevant for MLR.	Coefficients become unstable and uninterpretable.

Checking the LINE assumptions of Linear Regression

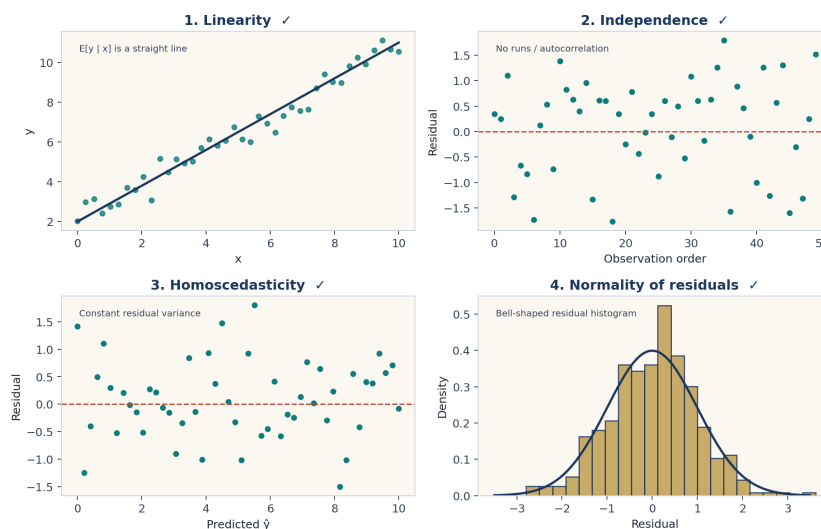


Figure 4. Healthy residual behaviour for the four LINE checks used in Part C.

4.1 How the notebook checks them

You do **not** check assumptions on raw y . You check them on **residuals after fitting**.

- **Residuals vs predicted** — random band around 0 $\Rightarrow$ linearity + homoscedasticity. A curve $\Rightarrow$ non-linearity. A funnel $\Rightarrow$ heteroscedasticity.
- **Histogram / KDE of residuals** — roughly bell-shaped $\Rightarrow$ normality is plausible.
- **Correlation heatmap** — very high $|r|$ between two features (e.g. bedrooms and bathrooms) flags multicollinearity.
- **Scatter of each x vs y** — a visual first look at linearity before any model is fit.

Residual plots as diagnostic tools

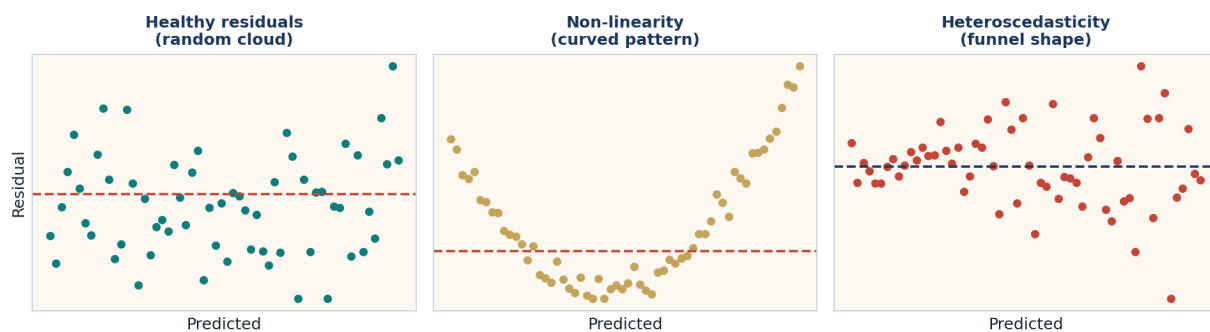


Figure 5. Residual patterns: a random cloud is healthy; a curve or funnel is a warning.

What you should write in the notebook interpretation

Do not blindly say “all assumptions hold”. Look at the plots. If residuals fan out for expensive houses, say so: “mild heteroscedasticity at the high end — RMSE will be driven by luxury outliers.” Honest diagnostics score higher than fake perfection.

05

Data, Features & Train-Test Split

Part B — preparing a supervised problem correctly

5.1 Independent vs dependent variables

The **dependent (target) variable** is what we want to predict. The **independent (feature) variables** are the inputs we are allowed to use.

Role	Column	Why
Dependent y	house_price_inr	The quantity the firm must estimate for a new listing.
Independent X	area_sqft, bedrooms, bathrooms, location_score, age_years, distance_city_km, lot_size_sqft, has_garage, has_pool, renovation_year	Measurable attributes of the house known before price is set.
Dropped	house_id	A pure identifier. It has no causal or predictive meaning and would only leak an index.

5.2 Why identifiers must be dropped

If house_id happens to correlate with price (newer listings get larger IDs, or IDs are sorted by locality), the model could “cheat” by using an ID pattern that will not exist for a brand-new house. Identifiers are not features.

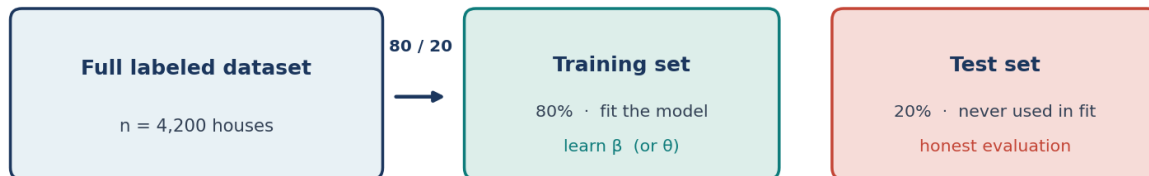
5.3 Binary and awkward columns

has_garage and **has_pool** are already 0/1 dummy variables — Linear Regression can use them directly. Their coefficients are rupee premia for having that amenity, holding other features fixed.

renovation_year may contain 0 for “never renovated”. As a raw number, 0 is not a year; it distorts the scale. A cleaner engineering choice (optional upgrade) is `years_since_renovation`, with a cap or a separate `never_renovated` flag. The baseline notebook uses the column as-is; mention this as a limitation in the report.

5.4 Train-test split

Train-Test Split (hold-out validation)



`random_state = 42` → reproducible split. Test data estimates real-world error.

Figure 6. An 80/20 hold-out split with a fixed `random_state` so results are reproducible.

We fit on the **training set** and report metrics only on the **test set**. Evaluating on training data is like marking an exam after the student has seen the answers — it cannot detect overfitting. A typical split is 80/20 (or 70/30). **random_state=42** freezes the shuffle so your classmate (and your future self) reproduces the same rows.

Never compute means, scalers, or `PolynomialFeatures` on the full dataset and then split. Fit transformers on **train only**, then transform test. Otherwise information from the test houses leaks into training — **data leakage** — and test scores become optimistically wrong.

Task 7-9 in one paragraph

Dependent variable: `house_price_inr`. Independent variables: all remaining columns after dropping `house_id`. Relationships are inspected with a correlation heatmap and scatter plots; `area_sqft` is expected to dominate. Data are split 80/20 with `random_state=42` so that later models are compared on an identical, unseen test fold.

06

Multiple Linear Regression

Many features, still linear in the parameters

6.1 The equation

Multiple Linear Regression (MLR) is Simple Linear Regression with p features instead of one. The model is still a **hyperplane**, still linear in β , and still estimated by OLS.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$$

$$y = X\beta + \varepsilon$$

X is the $n \times (p+1)$ design matrix (a column of 1s for the intercept, then the features). The OLS solution in matrix form is:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Many features, one price — Multiple Linear Regression

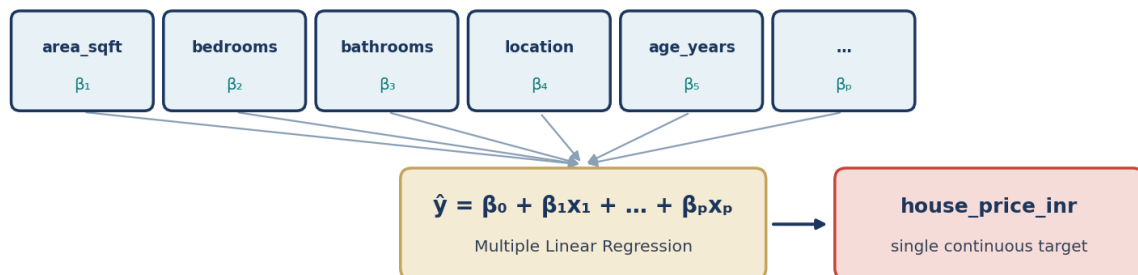


Figure 7. Each feature contributes an additive rupee effect; they combine into one price.

6.2 Reading a coefficient — ceteris paribus

β_k is the change in predicted price for a **one-unit increase in x_k** , **holding every other feature constant** (the Latin phrase is *ceteris paribus*). That is why MLR is so useful to a real-estate firm:

- $\beta(\text{area_sqft}) \approx$ extra rupees per extra square foot, for houses with the same bedrooms, location, age, ...
- $\beta(\text{has_pool}) \approx$ the average premium of having a pool, for otherwise similar houses.
- $\beta(\text{age_years})$ is usually negative — depreciation — unless “age” is actually prestige in a heritage pocket.
- $\beta(\text{distance_city_km})$ is usually negative — farther from the CBD, lower price.

6.3 Why MLR typically beats SLR here

Price is generated by many causes. SLR explains only the slice of variance that area shares with price. MLR adds bathrooms, location_score, amenities, and so on, so **R^2 rises and RMSE falls** — provided the extra features are relevant and not perfectly collinear. If you added a column of random noise, Adjusted R^2 would refuse to reward you (see Chapter 8).

On an Actual-vs-Predicted plot, a good MLR hugs the 45° line. Systematic bowing away from that line is leftover non-linearity — a hint that a transform or a polynomial term might help.

Task 17 — the sentence they want

Multiple Linear Regression outperforms Simple Linear Regression because it uses several relevant features and therefore captures more of the variance in house_price_inr. R^2 increases and error decreases. Performance would degrade only if extra features were irrelevant or highly collinear, which Adjusted R^2 and residual checks would reveal.

07

Polynomial Regression

Curves without leaving the linear-in- β family

7.1 The trick

Sometimes price does not grow in a straight line with area — returns may diminish, or luxury size may accelerate price. Polynomial Regression adds powers of x as extra columns:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_d x^d + \varepsilon$$

This looks non-linear in x , but it is **still linear in the parameters β** . That is why the notebook calls **PolynomialFeatures** and then fits ordinary **LinearRegression**. We did not invent a new optimiser; we expanded the feature space.

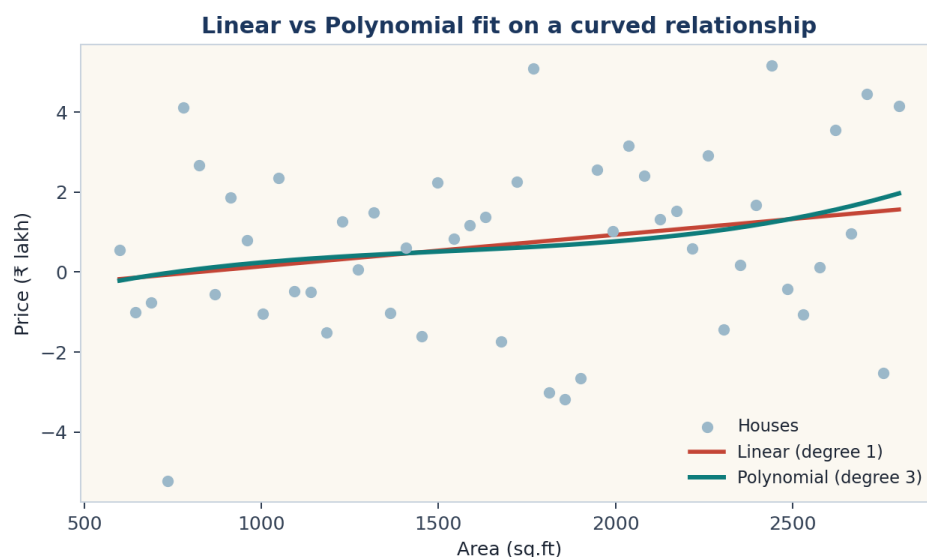


Figure 8. A degree-3 polynomial can follow gentle curvature that a straight line misses.

7.2 Degree 2 vs degree 3 — and the danger of degree 15

Degree	What the curve can do	Risk on this project
1 (SLR)	Straight line only	Underfitting if the true link is curved.
2 (quadratic)	One bend (U or inverted-U)	Usually a mild, safe extra flexibility.
3 (cubic)	Two bends	Can fit genuine curvature; watch the train-test gap.
High (e.g. 10+)	Wild oscillation, interpolates noise	Classic overfitting: great train R^2 , poor test R^2 .

In Part F the polynomial is fit on **area_sqft alone**, so it is a curved version of SLR, not a curved version of MLR. Even a beautiful cubic in area cannot know about pools or location. That is why a well-specified MLR often still wins on test R^2 .

Task 20 — spotting over / under-fit

If polynomial **training R^2 is high but test R^2 drops** → overfitting (high variance). If both linear train and test scores are low → underfitting (high bias). The best model has **high test R^2 and a small train-test gap**.

08

Model Evaluation Metrics

How we keep score in rupees and in variance explained

A model that “looks good” on a plot still needs a number. All metrics below are computed on the **test set** unless you are specifically diagnosing train vs test.

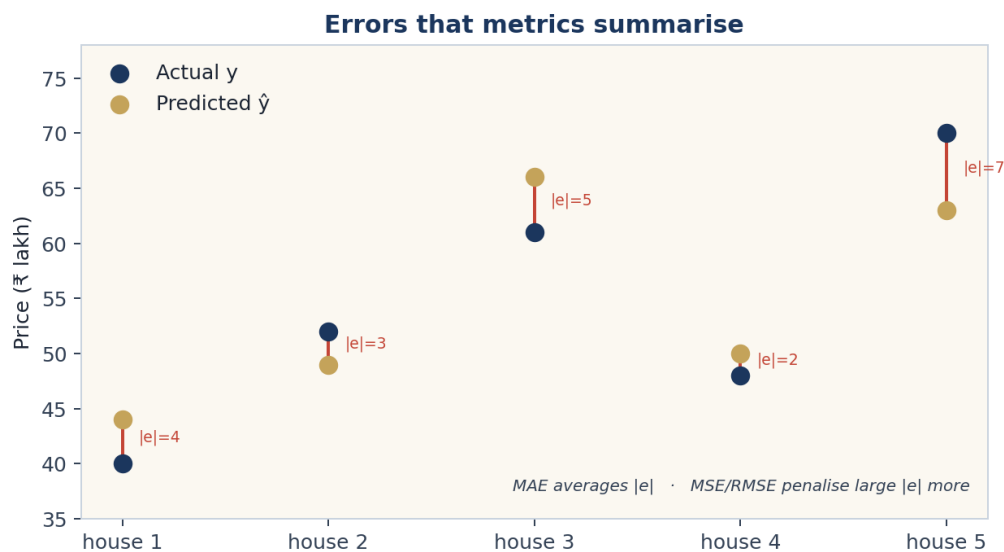


Figure 9. Every metric is a different summary of the same residuals $e_i = y_i - \hat{y}_i$.

8.1 Error metrics (lower is better)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE — Mean Absolute Error. Average rupee miss, equally weighting every house. The most client-friendly number: “we are typically off by ₹X.” Robust to outliers relative to MSE.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

MSE — Mean Squared Error. Squares the residual, so a ₹10 lakh miss costs 100 times a ₹1 lakh miss, not 10 times. Differentiable — this is the loss Gradient Descent minimises. Units are ₹², which humans cannot feel, so we rarely report MSE alone.

$$\text{RMSE} = \sqrt{\text{MSE}}$$

RMSE — Root MSE. Back in rupees, but still extra-punishing towards large errors. If $\text{RMSE} \gg \text{MAE}$, a few luxury houses dominate the error. Quote both.

8.2 Goodness-of-fit metrics (higher is better)

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

R² (coefficient of determination) is the fraction of variance in y that the model explains. R² = 1 is a perfect fit; R² = 0 means the model is no better than predicting $\bar{y}$ every time; R² can be negative if the model is worse than that baseline.

$$R_{adj}^2 = 1 - (1 - R^2) \frac{n-1}{n-p-1}$$

Adjusted R² inserts a penalty for extra features p. Ordinary R² never falls when you add a column (even noise). Adjusted R² rises only if the new feature improves fit enough to justify its degree of freedom. Always prefer Adj. R² when comparing SLR (p = 1) with MLR (p ≈ 10).

Metric	Units	Sensitive to outliers?	Use it to...
MAE	₹	Less	Talk to the business (“typical error”)
MSE	₹ ²	Very	Optimise (loss function)
RMSE	₹	Very	Report error in rupees with outlier emphasis
R ²	unitless (≤ 1)	Indirectly	Say how much variance is captured
Adj. R ²	unitless (≤ 1)	Indirectly	Compare models with different p

Reporting rule

Never crown a winner on training R². Rank models by **test RMSE and test Adj. R²**, then confirm the train-test gap is small. A model with R² = 0.93 train / 0.61 test has lost to a model with 0.81 / 0.79.

09

Bias-Variance, Overfitting & Underfitting

The central dilemma of every predictive model

9.1 Decomposing the error

For a squared-error loss, the expected test error of a model splits into three pieces:

$$E[(y - \hat{y})^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

- **Bias²** — error from wrong assumptions (a line forced through a curve). Underfitting. Falls as the model gets more flexible.
- **Variance** — error from the model twitching every time the training sample changes. Overfitting. Rises with flexibility.
- **σ² irreducible error** — noise in the real world (a buyer who overpays). No model removes it.

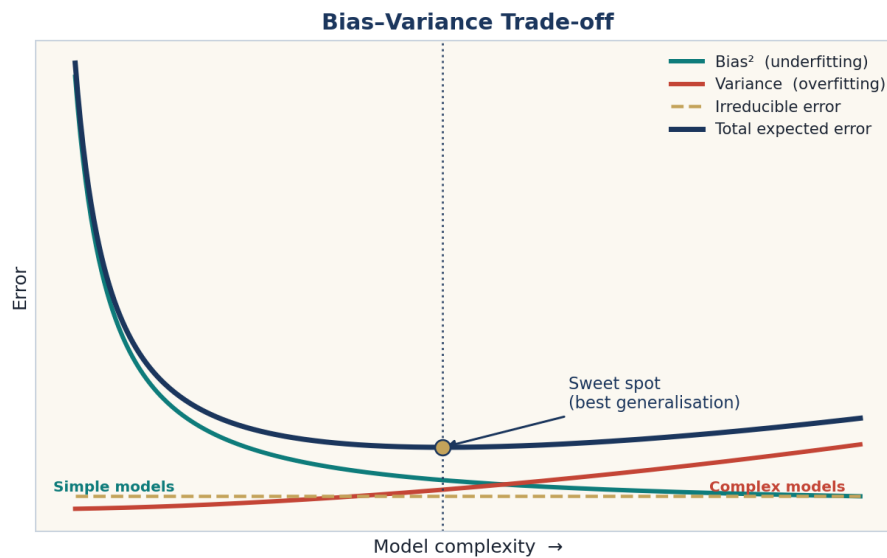


Figure 10. Total error is U-shaped in complexity. The optimum is the bottom of that U, not the fanciest curve.

9.2 Underfitting vs overfitting

Same data, three complexity levels

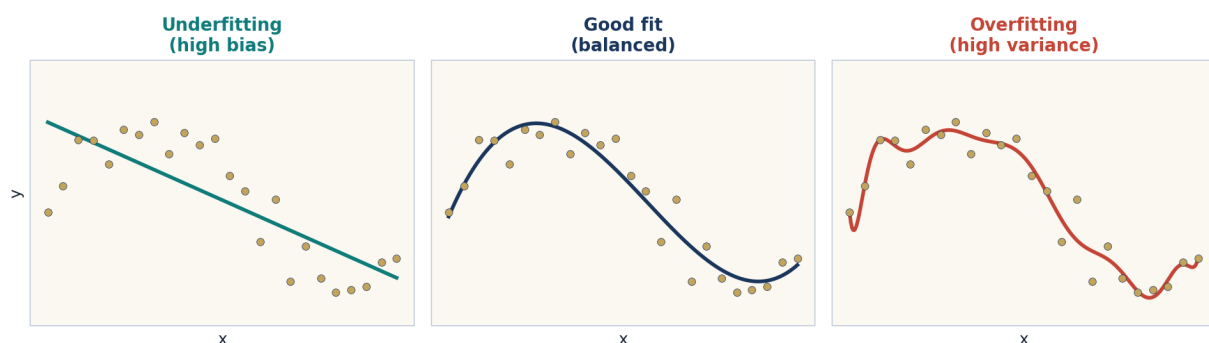


Figure 11. Same scatter, three models. Only the middle one will survive a new sample of houses.

	Underfitting (high bias)	Overfitting (high variance)
Symptom	Poor on train and test	Excellent on train, poor on test
Cause	Model too simple / too few features / too much regularisation	Model too complex / too few rows / trained too long
This project	Simple Linear Regression on area only	High-degree polynomial on area
Fix	Add features (MLR), add polynomial terms carefully, train longer	Reduce degree, add data, regularise (Ridge/Lasso), stop early
Train-test gap	Small, but both scores low	Large gap

9.3 The trade-off in one sentence

As model complexity increases, **bias falls and variance rises**. The model we deploy is the one that minimises **total expected error on unseen houses**, which is usually neither the simplest

nor the most wiggly. In this assignment that balance is typically **Multiple Linear Regression**.

Tasks 5, 6, 27, 28 — canned but correct

Bias = error from overly simple assumptions (underfitting). **Variance** = error from sensitivity to the training sample (overfitting). The trade-off is the sweet spot that minimises total error. Overfitting example: a high-degree polynomial that threads every training point. Underfitting example: a straight line through clearly curved (or multi-factor) price data. Best balance: high test R^2 and a small train-test gap.

10

Gradient Descent Optimisation

How a model walks downhill to the best parameters

10.1 Why we need it

OLS has a closed form, so why bother with Gradient Descent (GD)? Three reasons. (1) Closed-form OLS needs to invert $X^T X$, which costs $O(p^3)$ and breaks if features are collinear. (2) GD scales to huge n and to models with no closed form (neural nets). (3) The assignment asks you to **implement the idea from scratch** so you understand what scikit-learn is doing when it “fits”.

10.2 The picture

Imagine the cost $J(\theta)$ as a valley. You stand at a random θ , feel the slope ∇J (the gradient — direction of steepest ascent), and take a step the opposite way. Repeat until the ground is flat.

$$\theta := \theta - \alpha \nabla J(\theta)$$

α is the **learning rate**. Too large: you leap over the valley and diverge. Too small: you crawl and waste epochs. A well-chosen α produces a smoothly falling cost curve.

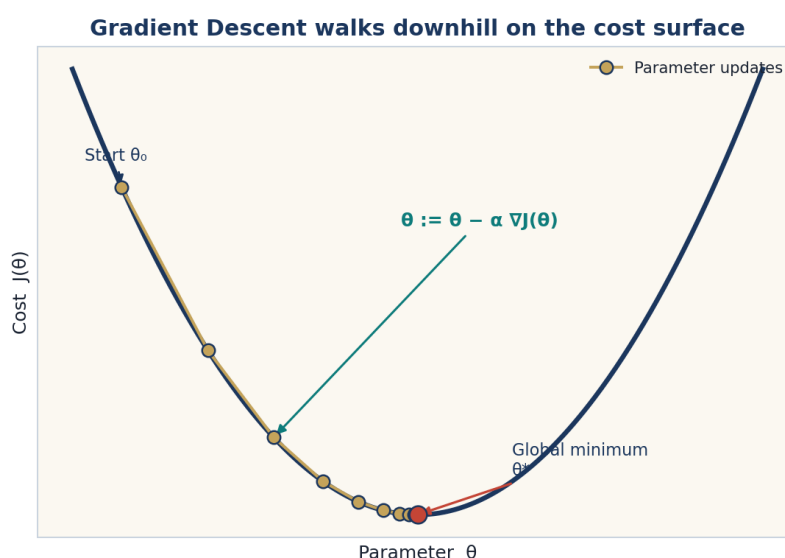


Figure 12. Each update subtracts α times the gradient, so iterates spiral toward θ^* .

For MSE linear regression the gradient has a neat form (m = number of training rows):

$$\nabla J(\theta) = \frac{2}{m} X^T (X\theta - y)$$

10.3 Feature scaling is not optional

area_sqft is in thousands; has_pool is 0 or 1. Without scaling, the cost surface is a long, skinny ravine: GD zig-zags and needs a tiny α . **StandardScaler** puts every feature on mean 0, variance 1 so the bowl is rounder and a single learning rate works. The notebook also scales y for numerical stability, then (in a full pipeline) would inverse-transform predictions back to rupees. Never fit the scaler on the test set.

10.4 Three flavours

Flavour	Gradient uses...	Behaviour	When to prefer
Batch GD	All m training rows every update	Smooth, stable path; one update per epoch; slow on large m	Small / medium data, when you want a textbook curve
Stochastic GD	One random row per update	Noisy path; many updates per epoch; can escape shallow minima	Huge data streams; online learning
Mini-batch GD	A small batch (e.g. 32 rows)	Mild noise, vectorised, GPU-friendly — the practical default	Almost all modern deep learning and large tabular jobs

Convergence behaviour of three Gradient Descent variants

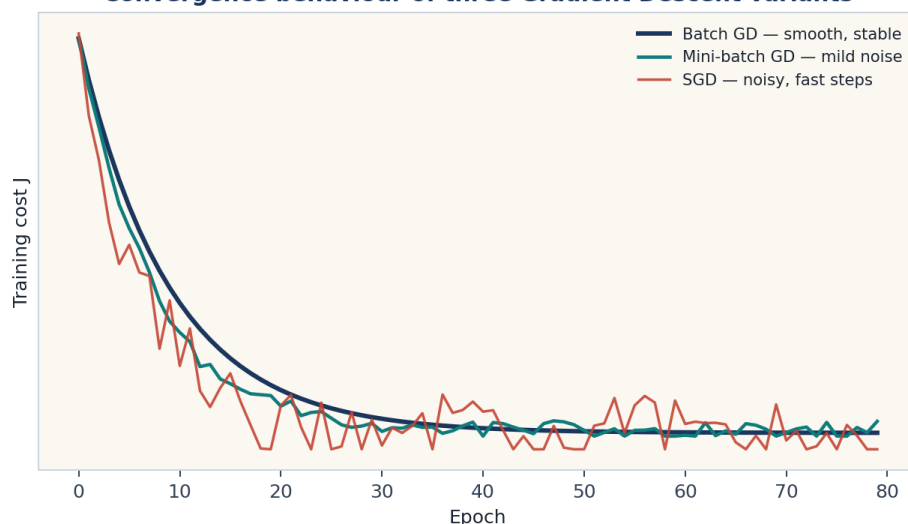


Figure 13. Batch is a slide; SGD is a jittery stagger; mini-batch is the compromise the industry uses.

In the notebook, cost is recorded **once per epoch** so the three curves are comparable. SGD's inner loop over individual rows makes wall-clock time per epoch larger in pure Python, even though each update is cheap. Mini-batch usually wins the speed/stability trade-off.

Tasks 21-25 — what to conclude

Batch GD: smoothest convergence, fewest surprises. SGD: fastest conceptual updates, noisy cost curve. Mini-batch GD: best practical balance of speed and stability, hence the choice for real-world scaling. Always show the three cost-vs-epoch plots and the three runtimes.

11**Diagnostics, Comparison & Business Meaning**

Part H-I — turning numbers into a recommendation

11.1 The comparison protocol

A professional comparison is not “which R^2 is biggest on whatever cell I just ran”. It is:

- Identical train/test split for every model (same `random_state`).
- Test MAE, RMSE, R^2 , Adj. R^2 in one table, sorted by test R^2 .
- Train R^2 beside test R^2 to read bias vs variance.
- At least one residual plot and one Actual-vs-Predicted plot for the winner.
- A business paragraph that names the price drivers, not only the metric.

11.2 What you should expect (and why)

Model	Bias / variance reading	Typical outcome on this task
Simple Linear Regression	High bias, low variance	Lowest test R^2 , highest RMSE — underfit
Polynomial (deg 2-3) on area	Lower bias than SLR, rising variance	Beats SLR if curvature exists; still blind to other features
High-degree polynomial	Low bias, high variance	Train R^2 inflates, test R^2 drops — overfit
Multiple Linear Regression	Best balance for a linear world	Usually highest test R^2 , lowest RMSE, small gap — the deployable model

11.3 Business interpretation

The firm does not buy an R^2 ; it buys a **trustworthy, explainable valuation**. MLR wins on those grounds as well as on metrics:

- A client can be told: “Your extra bathroom is associated with ₹ β more, holding area and location fixed.”
- Key drivers to highlight: **area_sqft, bathrooms, location_score** (and amenities if coefficients are large).
- Age and distance from the city typically drag price down — useful for discounting older suburban stock.
- A polynomial in area alone cannot answer “what is the pool worth?”; MLR can.
- A wildly overfit polynomial will misprice the next listing the moment it is slightly unlike the training sample — dangerous in a live valuation tool.

11.4 Honest limitations (good report material)

- Linear models miss interactions (a pool may be worth more in a hot climate / luxury locality).

- renovation_year = 0 is a data-quality issue, not a year.
- No temporal split: if the 4,200 rows span a boom and a crash, a random split leaks future prices into training.
- Irreducible error remains: interiors, legal title, seller urgency are not in X.
- Next upgrades: log(price) for skew, Ridge/Lasso for collinearity, tree ensembles if non-linearity dominates.

Final-report skeleton (Part I)

- 1. Best model:** Multiple Linear Regression — highest test R^2 , lowest RMSE, small train-test gap, interpretable coefficients.
- 2. Gradient descent:** Batch = smooth/slow; SGD = noisy/fast-updates; Mini-batch = the practical trade-off.
- 3. Fit quality:** SLR underfits; high-degree polynomial overfits; MLR balances.
- 4. Business:** Deploy MLR for quick, data-backed valuations driven by area, bathrooms and location score.

12

Formula Sheet & Glossary

One-page revision — write these from memory

12.1 Formulae you should be able to write

Simple Linear Regression	$y = \beta_0 + \beta_1 x + \varepsilon$
OLS slope	$\beta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$
OLS intercept	$\beta_0 = \bar{y} - \beta_1 \bar{x}$
Multiple Linear Regression	$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$
OLS (matrix)	$\hat{\beta} = (X^\top X)^{-1} X^\top y$
Polynomial (degree d)	$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_d x^d + \varepsilon$
MAE	$\text{MAE} = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $
MSE	$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
RMSE	$\text{RMSE} = \sqrt{\text{MSE}}$
R-squared	$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$
Adjusted R-squared	$R_{\text{adj}}^2 = 1 - (1 - R^2) \frac{n-1}{n-p-1}$
Bias-variance decomposition	$E[(y - \hat{y})^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$
Gradient descent update	$\theta := \theta - \alpha \nabla J(\theta)$
GD gradient for linear MSE	$\nabla J(\theta) = \frac{2}{m} X^\top (X\theta - y)$

12.2 Glossary of terms used in the project

Term	Meaning
Feature / independent variable	An input column X used to predict y .
Target / dependent variable	The column we predict — here <code>house_price_inr</code> .
Parameter / weight / coefficient	A number the algorithm learns (β or θ).
Hyperparameter	A choice we set, not learn: degree, learning rate α , batch size, <code>test_size</code> .
Residual e_i	$y_i - \hat{y}_i$. The signed rupee miss on house i .
Loss / cost J	A scalar the optimiser minimises (usually MSE).
Epoch	One full pass through the training data during GD.
Learning rate α	Step size of each GD update.
Generalisation	Doing well on houses not in the training set.
Data leakage	Test information sneaking into training (wrong scaler fit, ID features, future prices).
Dummy variable	A 0/1 encoding of a yes/no feature (<code>has_garage</code> , <code>has_pool</code>).
Ceteris paribus	“Other things equal” — how an MLR coefficient is interpreted.
BLUE	Best Linear Unbiased Estimator — OLS under the Gauss-Markov assumptions.
Hold-out set	The test (or validation) slice never used to fit parameters.
Multicollinearity	Features that move together, inflating coefficient variance.
Heteroscedasticity	Residual spread that changes with $\hat{y}$ (the funnel).
Irreducible error σ^2	Noise no model can remove, given these features.

13

Viva / Exam Quick Answers

Short, complete replies mapped to Parts A-I

Memorise the **bold lead sentence**, then expand with one project-specific example. Examiners mark definition + interpretation, not poetry.

Q1. What are supervised learning algorithms?

Algorithms that learn a mapping $X \rightarrow y$ from labelled pairs. Training uses known targets; inference predicts y for new X . This project is supervised because every training house already has `house_price_inr`.

Q2. Regression vs classification?

Regression predicts a continuous number (price in ₹). Classification predicts a discrete label (spam / not spam). Metrics, losses and plots all differ. We are in regression.

Q3. Explain Simple Linear Regression.

A straight line $\hat{y} = \beta_0 + \beta_1 x$ relating one feature to y . β_1 is ₹ per extra unit of x ; β_0 is the baseline. Parameters are chosen by OLS to minimise $\sum (y - \hat{y})^2$. We use `area_sqft`.

Q4. Assumptions of linear regression?

Linearity, Independence, Normality of residuals, Equal variance (homoscedasticity), and no multicollinearity in MLR. Checked with residual-vs- $\hat{y}$ plots, residual histograms, and a correlation heatmap.

Q5. Bias-variance trade-off?

Bias = error from a too-simple story (underfit). Variance = error from a too-flexible story that chases noise (overfit). Total error = $\text{Bias}^2 + \text{Variance} + \sigma^2$. We want the minimum of that sum, not zero bias at any cost.

Q6. Overfitting and underfitting with examples.

Overfitting: high-degree polynomial, train R^2 high, test R^2 low. Underfitting: SLR on multi-factor price data, train and test both weak. Good fit: similar, high scores on both splits — usually MLR here.

Q7. Independent vs dependent variables?

Dependent: `house_price_inr`. Independent: `area`, `bedrooms`, `bathrooms`, `location_score`, `age`, `distance`, `lot size`, `garage`, `pool`, `renovation_year`. `house_id` is dropped as an identifier.

Q8-9. Why visualise and why split?

Heatmaps/scatters reveal which features actually move with price and whether the link looks linear. An 80/20 split with `random_state=42` gives an honest, reproducible estimate of generalisation error.

Q10-12. SLR slope, intercept, residual plots.

Slope = rupees per extra sq.ft. Intercept = theoretical price at area 0. Residual-vs- $\hat{y}$ should be a random band around 0 (linearity + equal variance); residual histogram should be roughly bell-shaped (normality).

Q13-14. Why so many metrics?

MAE is typical ₹ error. RMSE is ₹ error that emphasises big misses. R^2 is variance explained. Adj. R^2 compares models with different numbers of features fairly. MSE is the optimisation loss.

Q15-17. Why does MLR change performance?

Extra relevant features explain extra variance, so R^2 rises and RMSE falls. Coefficients become partial effects. Collinear or useless features can undo the gain — hence Adj. R^2 .

Q18-20. Polynomial regression and overfit signs.

Add x^2 , x^3 , ... then run linear OLS. More bends, more risk. Train $\gg$ test R^2 means overfit; both low means underfit.

Q21. What is Gradient Descent?

An iterative optimiser: $\theta := \theta - \alpha \nabla J(\theta)$. It walks parameters downhill on the cost surface until the gradient is ~ 0 . Used when closed-form OLS is costly or unavailable.

Q22-25. Batch vs SGD vs Mini-batch.

Batch uses all rows $\rightarrow$ smooth, slower epochs. SGD uses one row $\rightarrow$ noisy, many updates. Mini-batch uses a handful (e.g. 32) $\rightarrow$ best speed/stability compromise. Compare cost-vs-epoch curves and wall-clock time.

Q26-28. How do you diagnose bias and variance?

Compare train R^2 vs test R^2 . Both low $\rightarrow$ high bias. Large gap $\rightarrow$ high variance. High and close $\rightarrow$ balanced. Complexity $\uparrow \Rightarrow$ bias $\downarrow$, variance $\uparrow$.

Q29. Best model and business takeaway?

Multiple Linear Regression: strongest test metrics, interpretable drivers (area, bathrooms, location), safe to generalise. Do not deploy an overfit polynomial. Mini-batch GD is the optimiser you would scale.

Last 30-second pitch

House price is a supervised regression problem. A single-feature line underfits; a wild polynomial overfits. Multiple Linear Regression, scored with test RMSE and Adj. R^2 , is the bias-variance sweet spot. Gradient Descent (especially mini-batch) is how we would estimate the same linear model when data grow large. The firm gets an explainable rupee estimate driven by size, amenities and location — not a black box.

End of notes · Predictive Insight Engine · Theory companion to the Jupyter notebook · Parts A-I

Use with the notebook: theory here, experiments there. Never quote a metric you did not compute.